

Data Transformation

Data transformation means converting raw data into a clean, usable format so that machine learning models can learn better.

1. Scaling

Definition: Rescales numerical features to a specific range or distribution.

What it does: It changes the magnitude of values without changing their shape.

Example: Convert values like [10, 20, 30] $\rightarrow$ [0.0, 0.5, 1.0] using Min-Max scaling.

Why use it: Many ML algorithms (like SVM, KNN, gradient descent models) are sensitive to feature magnitude.

2. Normalization

Definition: Transforms data so it follows a standard distribution or unit norm.

What it does: Often converts data to have mean = 0 and standard deviation = 1 (Z-score normalization).

Example: A dataset of heights is transformed so values become standardized like -1.2, 0.3, 1.8.

Why use it: Helps when features are on different scales and improves model stability and convergence.

3. Encoding Categorical Variables

Definition: Converts non-numeric categories into numeric form.

What it does: Turns text labels into numbers or binary vectors.

Example: Color: Red, Blue $\rightarrow$ One-hot encoding $\rightarrow$ Red = [1,0], Blue = [0,1]

Why use it: ML models only understand numerical input, not text.

4. Binning

Definition: Groups continuous values into discrete intervals.

What it does: Converts continuous data into categories (bins).

Example: Age: 0–18 = Child, 19–60 = Adult, 60+ = Senior

Why use it: Reduces noise and makes patterns easier to interpret.

5. Logarithmic Transformation

Definition: Applies a log function to reduce skewness in data.

What it does: Compresses large values and expands small ones.

Example: Income: [100, 1000, 100000] $\rightarrow$ [2, 3, 5]

Why use it: Helps handle highly skewed distributions and outliers.

6. Polynomial Feature Mapping

Definition: Creates new features by raising existing features to powers or combining them.

What it does: Adds non-linear relationships to linear models.

Example: $x \rightarrow x^2, x^3$, or $x_1 \times x_2$

Why use it: Helps linear models learn non-linear patterns.

7. Feature Engineering

Definition: Creating new meaningful features from raw data.

What it does: Transforms or combines existing data into more useful representations.

Example: From Date $\rightarrow$ extract Day, Month, Year

Why use it: Improves model performance by providing better information.

8. Clipping and Winsorizing

Definition: Limits extreme values to reduce the effect of outliers.

What it does: Caps values above or below a threshold.

Example: If max threshold = 100, values [20, 50, 500] $\rightarrow$ [20, 50, 100]

Why use it: Prevents outliers from dominating model training.

9. Discretization

Definition: Converts continuous variables into discrete categories.

What it does: Similar to binning but often more formal or model-driven.

Example: Temperature $\rightarrow$ Cold, Warm, Hot

Why use it: Simplifies models and improves interpretability.

10. Frequency Domain Transformation

Definition: Converts data from time/space domain into frequency domain.

What it does: Extracts frequency patterns using transforms like Fourier Transform.

Example: Audio signal $\rightarrow$ frequency spectrum showing pitch components.

Why use it: Useful for signals, images, and time-series analysis where patterns exist in frequency.

Scaling & Normalization

Scaling:

- Makes all features use a similar range of values
- Example: Salary (0–100,000) and Age (0–100) → both converted to something like 0–1.
- So no feature looks more important just because its numbers are bigger.

Normalization:

- Adjusts each data point so its total size becomes 1.

Two vectors:

$$A = [3,4]$$

$$B = [30,40]$$

- They point in the same direction (same ratio 3:4), but B is just bigger.
- Without normalization: model thinks they are very different
- With normalization: both become [0.6,0.8]

Now the model understands:

“These are essentially the same pattern.”

Keeps the ratio between values but removes the effect of overall size.

Scaling Techniques

Min-Max Scaling

This is like squashing a whole ruler down to fit between **0 and 1**.

$$\text{Scaled Feature, } X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

What it does

- Smallest value $\rightarrow 0$
- Largest value $\rightarrow 1$
- All other values $\rightarrow$ between 0 and 1

Example

Feature: [10,20,30]

min = 10, max = 30

After scaling: [0,0.5,1]

Why use it?

- Keeps relative differences between values
- Works well when you need data in a bounded range (e.g., neural networks)
- Easy to interpret

Limitation

- Sensitive to outliers, If one value is extremely large, most data gets squeezed near 0

2. Standardization (Z-Score)

This is a bit more sophisticated. Instead of squashing everything into a box, it centers the data around the **Average (Mean)**.

$$\text{Standardized Feature, } X' = \frac{X - \mu}{\sigma}$$

where, μ = mean & σ = Standard Deviation

It transforms data so that:

- mean = 0
- standard deviation = 1

Formula

- **x = original value**
- **μ = mean/Average** (the middle of the pack).
- **σ = Standard Deviation** (how much the data "spreads out").

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

What it does

- Subtracting $\mu \rightarrow$ centers data around 0
- Dividing by $\sigma \rightarrow$ scales spread to unit variance

So values are expressed as “how far a value is from the average, in standard deviation units”

Example

If mean = 50, std = 10:

- $x=60 \rightarrow z=+1$ (above average)
- $x=40 \rightarrow z=-1$ (below average)

Why use it?

- It's great when your data has "outliers" (those weirdly high or low numbers), because it doesn't force them into a 0-to-1 box.
- Makes features comparable
- Helps models converge faster
- Important for algorithms like Logistic Regression, SVM, Neural Networks

3. MaxAbs Scaling

$$x' = \frac{x}{\max(|x|)}$$

What it does

- Brings all values into the range [-1, 1]
- Keeps zero-centered data unchanged
- Preserves the sign (positive/negative)

Example

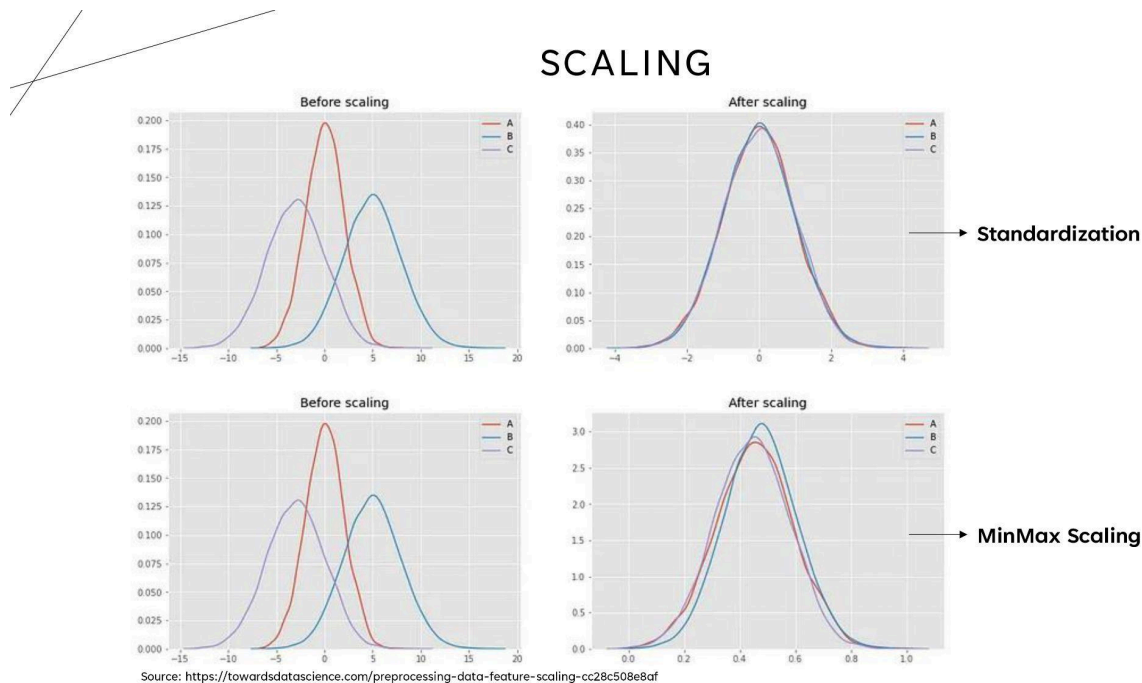
Feature: [-5,0,10]

Max absolute value = 10

After scaling: [-0.5,0,1]

Why use it?

- Useful when data already has both positive and negative values
- Does not shift data (unlike z-score, which changes mean)
- Good for sparse data (e.g., text features)



Standardization

Look at the top-right graph. What changed?

- **The Center:** All three curves (A, B, and C) are now perfectly centered over 0.
- **The Spread:** They are all roughly the same width now.
- **The Scale:** If you look at the bottom axis, the numbers now go from about -4 to 4.
- **Why it works:** By subtracting the average and dividing by the spread, we've forced every group to "line up" in the middle. They still keep their original "bell" shape, but they are now speaking the same language.

Min-Max Scaling

Now look at the bottom-right graph. This looks a bit different:

- **The Boundaries:** Look at the bottom axis. Notice how it starts exactly at 0.0 and ends exactly at 1.0?
- **The "Squash":** All three curves have been shoved into this tiny box between 0 and 1.
- **Why it works:** This is great for when you need a very strict limit. No matter how wild the data was before, it is now trapped in that 0-to-1 range.

Normalization Techniques

L₁ Normalization (Manhattan Norm)

$$X' = \frac{X}{\sum |X|}$$

It rescales each data point so that the sum of absolute values = 1.

What it does

- Adds up absolute values of a vector
- Divides each element by that sum
- Result: total becomes 1

Example

Vector: [2,3,5]

Sum = $2+3+5 = 10$

After normalization: [0.2,0.3,0.5]

Now, $0.2+0.3+0.5 = 1$

Why use it?

- Emphasizes proportions (like percentages)
- Useful in text data (TF, probabilities)
- Makes values easier to compare within a data point
- Larger values get relatively more weight, smaller ones less - but not zero

L₂ Normalization (Euclidean Norm)

$$\text{Normalized Feature, } X' = \frac{X}{\sqrt{\sum X^2}}$$

It rescales each data point so that its length (magnitude) = 1.

What it does

- Computes the vector length using squares
- Divides each value by that length
- Result: vector has unit length = 1

Example

Vector: [3,4]

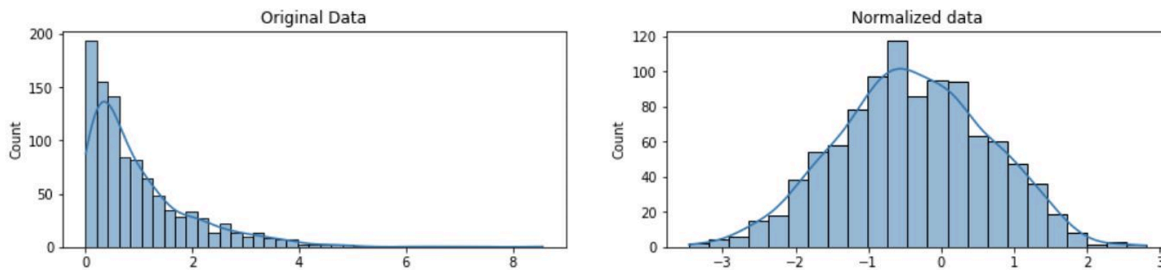
Length: $\sqrt{3^2+4^2} = 5$

After normalization: [0.6,0.8]

Now, $\sqrt{0.6^2+0.8^2} = 1$

Why use it?

- Focuses on direction (pattern), not magnitude
- Useful in cosine similarity, text analysis
- Makes different-sized data points comparable



Look at the two charts,

- **Original Data:** Notice how the bars are all bunched up on the left (near 0) and then there's a long "tail" stretching out to the right (near 8). This is called "skewed" data. It's messy and hard for a model to learn from because most of the action is in one corner.
- **Normalized Data:** Look at the chart on the right. After we applied the math, the data transformed into a beautiful **Bell Curve** (a normal distribution).
 - The center is now near **0**.
 - The data spreads out evenly to the left and right (between roughly **-3 and 3**).

Categorical Variables: Encoding

Label Encoding

Converts categorical labels into numbers.

What it does:

Each unique category is assigned an integer

Feature: ["Red", "Green", "Blue"]

After encoding: [0,1,2]

Important

- The numbers do NOT mean order or importance
- But some models may mistakenly assume order (e.g., $2 > 1 > 0$)

When to use it?

- Good for target labels (e.g., classification output)
- Can be used for input features only if the categories are ordinal (have natural order)

Original Data			Label Encoded Data	
Team	Points		Team	Points
A	25	→	0	25
A	12		0	12
B	15		1	15
B	14		1	14
B	19		1	19
B	23		1	23
C	25		2	25
C	29		2	29

Source: <https://www.statology.org/label-encoding-vs-one-hot-encoding/>

One Hot Encoding

Converts each category into a binary vector (0s and 1s).

What it does

- For each category, create a new column
- Only one column = 1, rest = 0

Example

Feature: ["Red", "Green", "Blue"]

After encoding:

- Red → [1,0,0]
- Green → [0,1,0]
- Blue → [0,0,1]

Why use it?

- Avoids false ordering (unlike label encoding)
- Works well with most ML models

Limitation

- Creates many columns if categories are large (high dimensionality)

Original Data		One-Hot Encoded Data			
Team	Points	Team_A	Team_B	Team_C	Points
A	25	1	0	0	25
A	12	1	0	0	12
B	15	0	1	0	15
B	14	0	1	0	14
B	19	0	1	0	19
B	23	0	1	0	23
C	25	0	0	1	25
C	29	0	0	1	29

Source: <https://www.statology.org/label-encoding-vs-one-hot-encoding/>

Hyperparameters

Hyperparameters are the external parameters set before the learning process of a machine learning model begins. They influence the training process, determining how the model learns, but they are not learned by the model itself. Unlike model parameters (such as weights in linear regression/neural networks), hyperparameters are defined by the practitioner and need to be tuned manually or via techniques like grid search, random search, or more advanced optimization methods like Bayesian optimization.

Types of Hyperparameters

- **Model-Specific:** Choice of Algorithm, Number of layers and Architecture (in Neural Networks), Number & Max depth of trees (in Random Forest), Degree of Polynomials (in Regression), etc.
- **Training-Specific:** Choice of Optimizing Algorithm, Cost Function, Learning Rate (α), Batch Size, Number of Epochs, etc.
- **Regularization Hyperparameters:** Choice of Regularizer (L1/L2/Dropout etc.), Regularization Parameter (λ), Dropout Rate, etc.

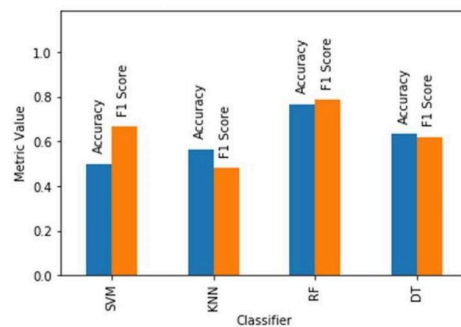
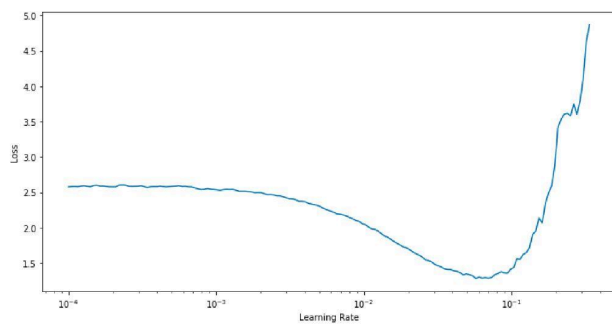
- Data-Specific: Choice of Feature Scaling technique, Extent of data augmentation (Rotation Angle, Noise Level, etc.).

Hyperparameter Tuning (Manual)

Manual tuning means changing one hyperparameter at a time and checking model performance on validation data.

- Try different values
- Plot performance (accuracy/loss) vs values
- Pick the value that gives the best validation result

This approach is simple but effective for tuning one hyperparameter at a time when computational resources are limited.



Early Stopping

Early stopping is a technique that automatically stops training when the model stops improving on the validation set.

How it works

- Monitor validation performance during training
- If it does not improve for a set number of epochs (patience) → stop training
- Restore the best model weights seen so far

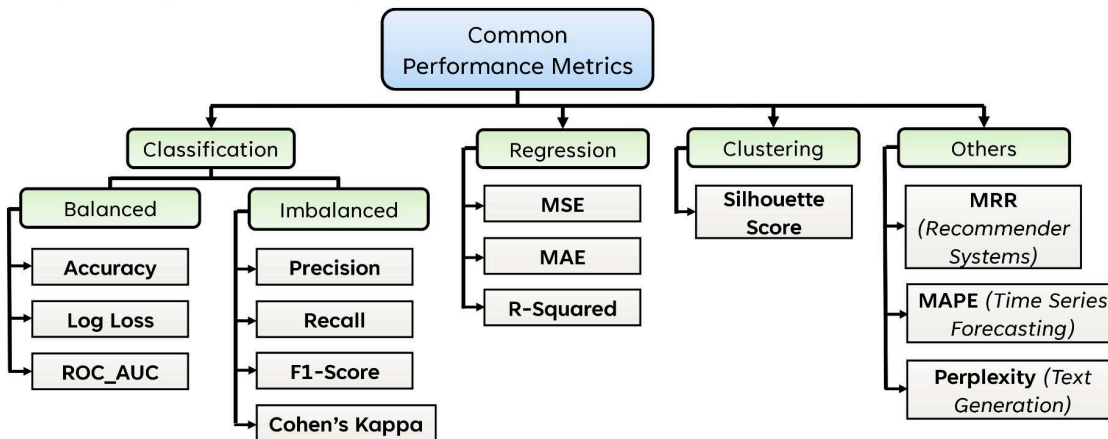
Why it is used

- Prevents overfitting
- Saves training time and computation

- Keeps the model at its best generalization point
- It act as an implicit hyperparameter optimization

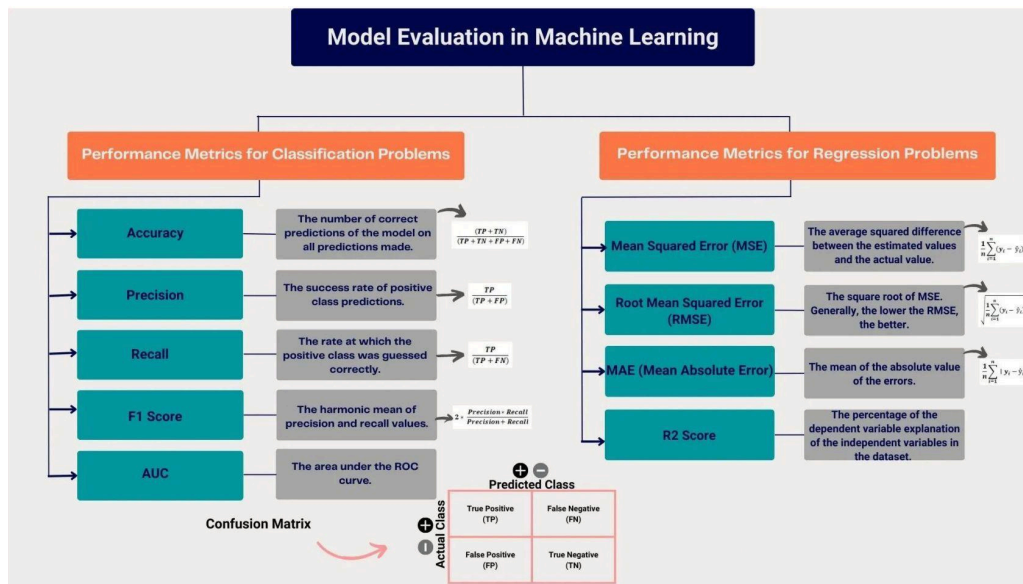
EVALUATION: PERFORMANCE METRICS

ML **Performance Metrics** are used to **evaluate** how well a model performs on a specific task. These metrics provide quantitative insights into the model's accuracy, generalization, and robustness. The choice of metric depends on the type of problem (classification, regression, etc.) and the goals of the analysis.



67

PERFORMANCE METRICS



Accuracy & Error Rate

Accuracy

It tells how many predictions are correct out of all predictions.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Where,

- TP (True Positives): Correct predictions of the positive class.
- TN (True Negatives): Correct predictions of the negative class.
- FP (False Positives): Incorrect predictions of the positive class (Type I Error).
- FN (False Negatives): Incorrect predictions of the negative class (Type II Error).

Higher accuracy = better model performance

In multi-class classification, when evaluating one class, we treat that class as positive (1) and all other classes together as negative (0). So we repeatedly do a one-vs-rest evaluation for each class.

Key problems:

- Ignores class imbalance: If 95% data is class 0, a model predicting only 0 gets 95% accuracy but is useless.
- Poor for critical tasks: In medical or fraud detection, missing a positive case can be serious, but accuracy may still look high.

Error Rate

It tells how many predictions are wrong.

$$\text{Error Rate} = \frac{FP+FN}{TP+TN+FP+FN} = 1 - \text{Accuracy}$$

Lower error rate = better model performance

Balanced vs Imbalanced Classes

- Balanced dataset: All classes have a similar number of samples.
- Imbalanced dataset: One class has many more samples than others.

Problem with imbalance

- Model becomes biased toward majority class
- It learns to predict the frequent class more often
- Accuracy can look high but performance is actually poor

Example

- 95% Class A, 5% Class B:
- The model always predicts A → 95% accuracy, but it never detects B.

Why it matters

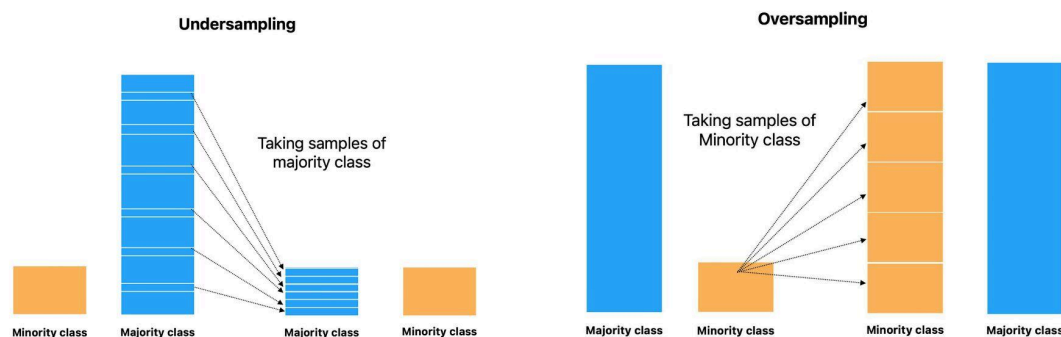
- Minority classes are often the most important (fraud detection, disease detection)
- Missing them can be costly or dangerous

Better evaluation

Instead of only accuracy, use: Precision, Recall, F1-score

Handling imbalance

- Oversampling: Increases the number of samples in the minority class (by duplicating or generating new data) to balance the dataset.
- Undersampling: Reduces the number of samples in the majority class to match the minority class, which may lead to loss of information.



Evaluating Models on Imbalanced Data

For imbalanced datasets, accuracy is not enough. We use better metrics that look at class-wise performance.

Precision

$$\text{Precision} = \frac{TP}{TP+FP}$$

Of all predicted positives, how many are actually correct. Important when false positives are costly (e.g., spam detection)

Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP+FN}$$

Of all actual positives, how many were correctly found. Important when missing positives is costly (e.g., disease detection)

F1-Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Balance between precision and recall. Useful when you need both low false positives and low false negatives

Adjusting the Threshold

In binary classification, models output a probability (0 to 1). The threshold decides how this probability becomes a class label.

Default rule

- If probability $\geq 0.5 \rightarrow$ Positive (1)
- If probability $< 0.5 \rightarrow$ Negative (0)

Changing the threshold

Higher threshold (e.g., 0.7):

- Model becomes more strict
- Fewer positives predicted
- Higher precision, lower recall
- Fewer false alarms

Preferred cases:

- Spam detection (email filtering): You don't want important emails wrongly marked as spam.
- Banking transactions: Incorrectly flagging a normal transaction can annoy users or block legitimate payments.
- Autonomous systems / security alerts: Too many false alarms reduce trust in the system.
- Recommendation systems (strict filtering): Only show very confident recommendations to avoid poor suggestions.

Lower threshold (e.g., 0.3):

- Model becomes more lenient
- More positives predicted
- Higher recall, lower precision

Preferred cases:

- Medical diagnosis (e.g., cancer, disease screening): Better to flag more possible cases than miss a real patient.
- Fraud detection (initial screening stage): Catch as many suspicious cases as possible for further review.
- Security systems (intrusion detection): It's safer to investigate extra alerts than miss a real attack.
- Search and recommendation recall-focused systems: Prefer showing more relevant results, even if some are less relevant.

AUC-ROC

AUC-ROC is a metric that evaluates how well a classification model separates positive and negative classes across all thresholds.

ROC Curve

It plots:

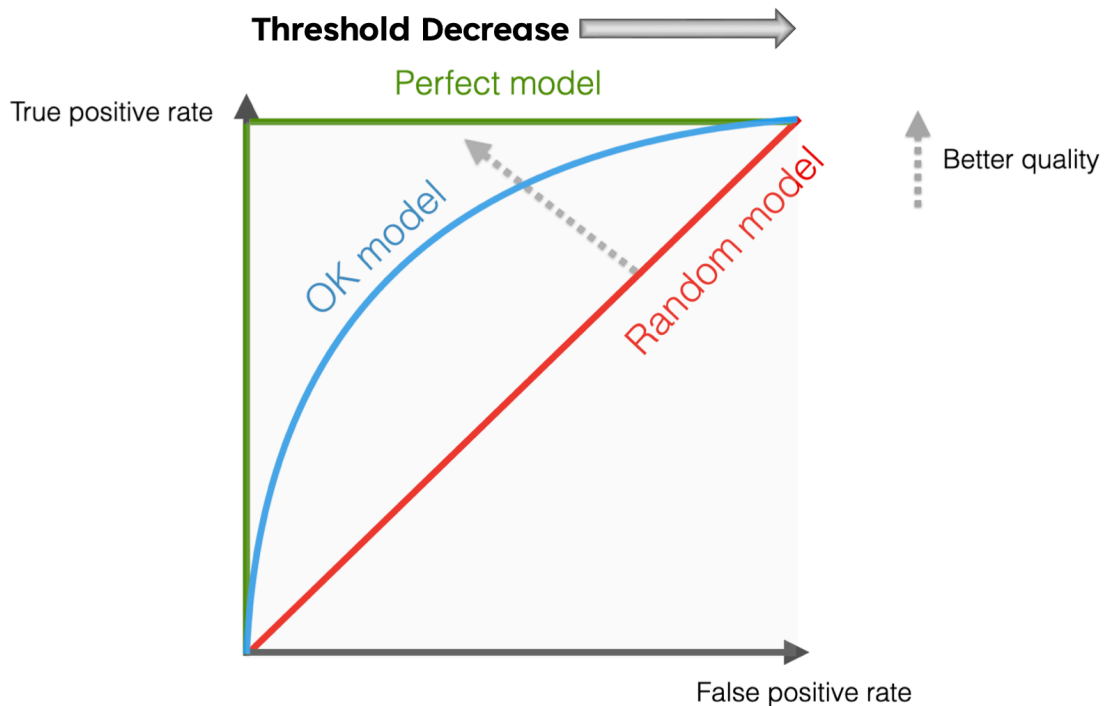
TPR (Recall) on Y-axis

$$\text{TPR} = \frac{TP}{TP + FN}$$

FPR on X-axis

$$\text{FPR} = \frac{FP}{FP + TN}$$

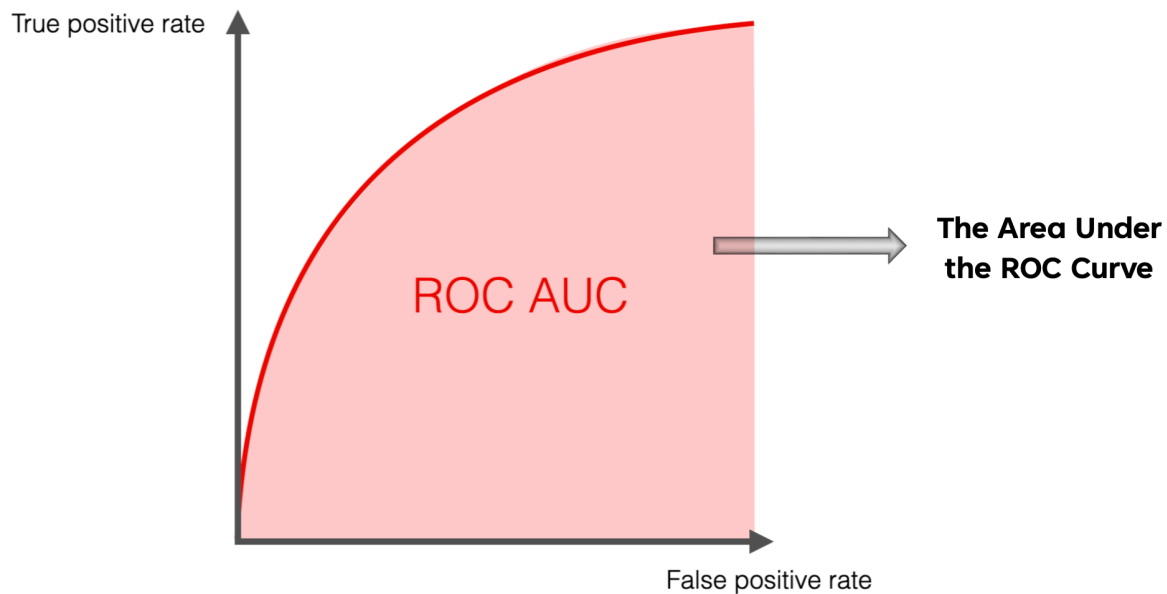
A curve closer to the top-left corner indicates a better model, meaning higher sensitivity and lower false positives. If the curve is closer to the diagonal line (random guessing line), it indicates poor performance.



AUC (Area Under Curve)

- Measures the overall ability of the model to distinguish classes
- Value ranges from 0 to 1

A ROC AUC score or AUC-ROC is a single metric to summarize the performance of a classifier across different thresholds. To compute the score, we must measure the area under the ROC curve.



Interpretation

- $AUC = 1 \rightarrow$ perfect separation of classes
- $AUC = 0.5 \rightarrow$ random guessing
- $AUC < 0.5 \rightarrow$ worse than random
- $AUC > 0.5 \rightarrow$ model has meaningful learning ability

Key intuition

- ROC shows performance at different thresholds
- AUC summarizes it into a single score

Confusion Matrix

A confusion matrix is a table that compares actual labels vs predicted labels to evaluate a classification model.

Structure (binary case)

- TP (True Positive): correct positive predictions
- TN (True Negative): correct negative predictions
- FP (False Positive): wrong positive predictions
- FN (False Negative): wrong negative predictions

Why it is useful

- Shows how many predictions are correct and incorrect for each class
- Helps calculate key metrics like: Precision, Recall, Accuracy
- Reveals types of errors (FP vs FN)

Unlike accuracy, it shows where the model is failing, not just how often it is correct.

CONFUSION MATRIX

Binary Classification

Actual	Positive	23	7
	Negative	10	60
		Positive	Negative
		Predicted	

Multi-Class Classification

Actual	Elephant	25	3	0	2
	Monkey	3	53	2	3
	Fish	2	1	24	2
	Lion	1	0	2	71
		Elephant	Monkey	Fish	Lion
		Predicted			

For a **binary confusion matrix**, the positions are fixed like this:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

So from your image:

	Predicted Positive	Predicted Negative
Actual Positive	23	7
Actual Negative	10	60

Therefore:

- **TP (True Positive) = 23**
→ Actual positive and predicted positive
- **FN (False Negative) = 7**
→ Actual positive but predicted negative
- **FP (False Positive) = 10**
→ Actual negative but predicted positive
- **TN (True Negative) = 60**
→ Actual negative and predicted negative

A quick memory trick:

- **True** = prediction is correct
- **False** = prediction is wrong

- **Positive/Negative** = predicted class

So:

- TP → predicted positive correctly
- TN → predicted negative correctly
- FP → predicted positive wrongly
- FN → predicted negative wrongly

For **multi-class confusion matrices**, choose one class at a time as the “positive” class and treat all others as “negative”.

Example: take **Elephant** as positive.

Matrix row/column for Elephant:

Actual \ Predicted	Elephant	Monkey	Fish	Lion
Elephant	25	3	0	2
Monkey	3	53	2	3
Fish	2	1	24	2
Lion	1	0	2	71

For class **Elephant**:

- **TP = 25**
(Actual Elephant, Predicted Elephant)
- **FN = 3 + 0 + 2 = 5**
(Actual Elephant but predicted something else)

- **FP = 3 + 2 + 1 = 6**
(Predicted Elephant but actually another class)
- **TN = all remaining cells**
 $53+2+3+1+24+2+0+2+71=158$

So for Elephant:

- TP = 25
- FN = 5
- FP = 6
- TN = 158

Loss & Accuracy Curves

Loss and accuracy curves show how a model's performance changes during training over epochs.

Loss Curve

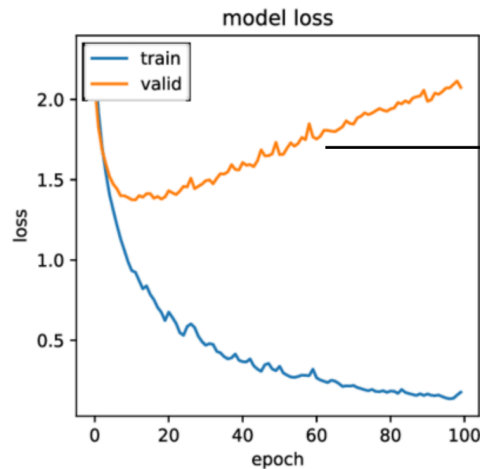
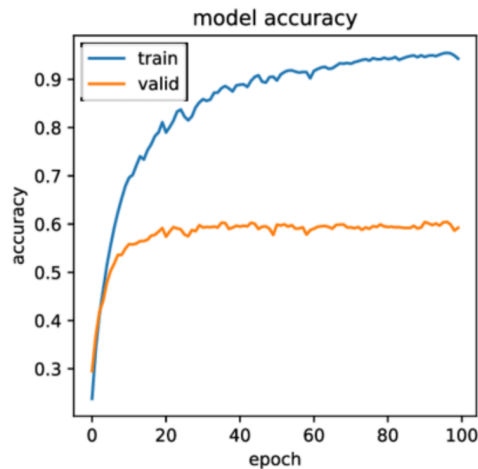
- Shows how the error (cost) changes
- Should generally decrease over time
- Lower loss = better model fit

Accuracy Curve

- Shows how many predictions are correct
- Should generally increase over time
- Higher accuracy = better performance

Why they are important

- Help track training progress
- Detect:
 - Overfitting (training improves, validation worsens)
 - Underfitting (both stay poor)
- Show whether the model is learning properly



The Model is Overfitting

PRACTICAL SCENARIO 1

Living Area (feet ²)	#Bedrooms	Price (X1000\$)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	516

The task is to **predict house prices** based on the living area & the number of Bedrooms (multiple features) using the above dataset.

- ☐ What are the **Data Transformation** techniques that can be used to prepare data for this task?
- ☐ What are the **Hyperparameters** that may need **tuning** (based on on you chosen model)?

The task is predicting house prices based on two features: Living Area (in square feet) and the Number of Bedrooms. Because the target variable (Price) is a continuous number, this is a regression problem.

1. Data Transformation Techniques

The key step is feature scaling because the features are on different scales (thousands vs single digits), which can bias the model.

Common methods:

- Standardization (Z-score)
- Min-Max Scaling

- MaxAbs Scaling

2. Hyperparameters to Tune Hyperparameters are set before the learning process begins and vary heavily depending on the algorithm we choose to build your model. Depending on the regression model chosen for this task, we might tune the following:

Random Forest

- Number of trees
- Max depth

Neural Network

- Layers and neurons
- Learning rate, batch size, epochs
- Optimizer and loss function
- Regularization (L1/L2, λ)

PRACTICAL SCENARIO 2

Order_ID	Customer_Name	Product	Quantity	Price	Date	Total_Sales	Target
101	John Doe	Product A	2	20.5	1/1/2023	41	Successful
102	jane doe	Product B	1	35		35	Unsuccessful
103	MARY LOU	Product C	5		1/3/2023	175	Successful
104	John Doe	Product A	-3	20.5	1/4/2023	-61.5	Unsuccessful
105	Mary Lou	Product B	2	35	1/5/2023	70	Successful
106		Product D	1	50	1/6/2023	50	Successful
107	jane DOE	Product C	10	17.5	1/7/2023	175	Successful
108	Mary Lou	Product A	2		1/8/2023		Unsuccessful
109	John Doe	Product B	1	35	1/9/2023	35	Successful
110		Product D	10		10-Jan	2023	Unsuccessful
111	jane doe	Product C	5	17.5	1/11/2023	87.5	Successful
112	John DOE	Product A	1		1/12/2023		Unsuccessful

- ☐ What are some **Data Cleaning** techniques that you may need to clean this dataset & Why?
- ☐ What are some **EDA** techniques that you may need to analyze this dataset?
- ☐ What are some **Data Transformations** that you may need to prepare this dataset?
- ☐ What kind of ML model would you like to use to perform this Binary Classification task (Whether a sales transaction is **successful** based on the features in the dataset)?

3. To prepare the dataset for ML:

One-Hot Encoding:

- Apply to Product (A, B, C, D) since it is nominal with no order.

Label Encoding:

- Convert Target into binary values (Successful = 7, Unsuccessful = 5).

Feature Scaling:

Apply Standardization or Min-Max Scaling to:

- Quantity
- Price
- Total_Sales

This prevents large values (like Total_Sales) from dominating.

Feature Engineering:

Extract useful features from Date:

- Day
- Month
- Year

4. Since the goal is to predict Successful vs Unsuccessful, this is a binary classification task.

Possible Models:

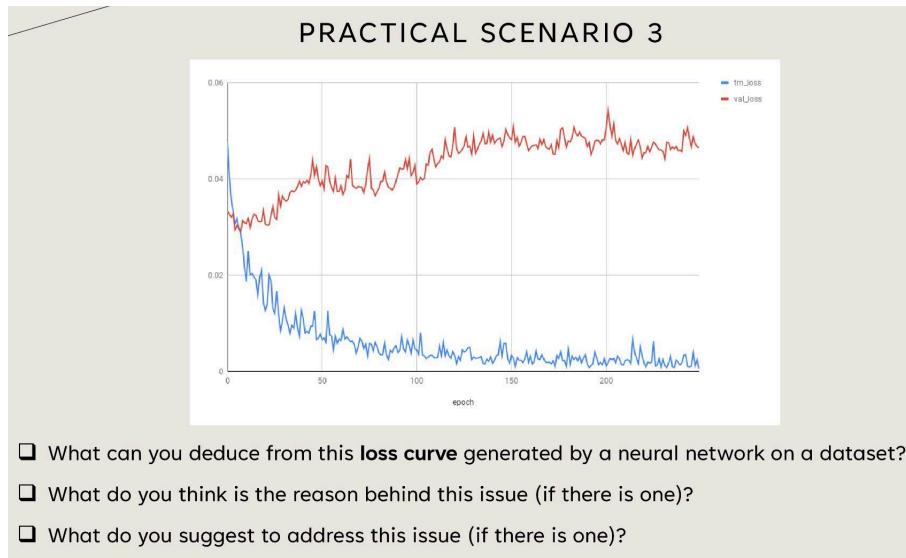
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Random Forest (RF)
- Decision Tree (DT)
- Logistic Regression
- Neural Networks

Evaluation Metrics:

- Accuracy
- Precision
- Recall

- F1-Score
- AUC-ROC

Since classes are imbalanced, we should prioritize F1-Score, Recall, Precision, or AUC-ROC over Accuracy for better evaluation.



1. Observation from Loss Curve

The model is overfitting.

- Training loss keeps decreasing toward zero
- Validation loss decreases at first, then steadily increases

This shows the model performs well on training data but poorly on unseen data.

2. Reason for the Issue

The model is trained for too long and becomes too complex.

It starts to memorize the training data (including noise) instead of learning general patterns, which leads to poor performance on the validation set.

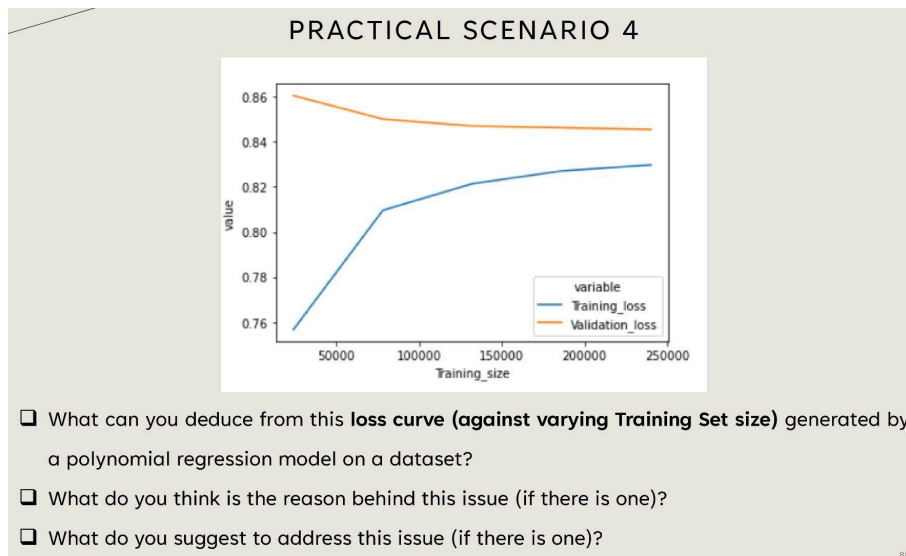
3. Solutions to Fix Overfitting

Early Stopping: Stop training when validation loss stops improving for a set number of epochs (patience), and keep the best model.

Regularization: Reduce model complexity using:

- L1 / L2 regularization (λ)
- Dropout layers
- Adjusting dropout rate

These methods help the model generalize better to unseen data.



1. Observation from Loss Curve

The training and validation loss curves are converging.

- The gap between training and validation loss is small
- Both losses flatten at a relatively high value
- Increasing training data does not significantly reduce validation loss

This suggests the model is not improving further.

2. Reason for the Issue

This pattern indicates underfitting (high bias).

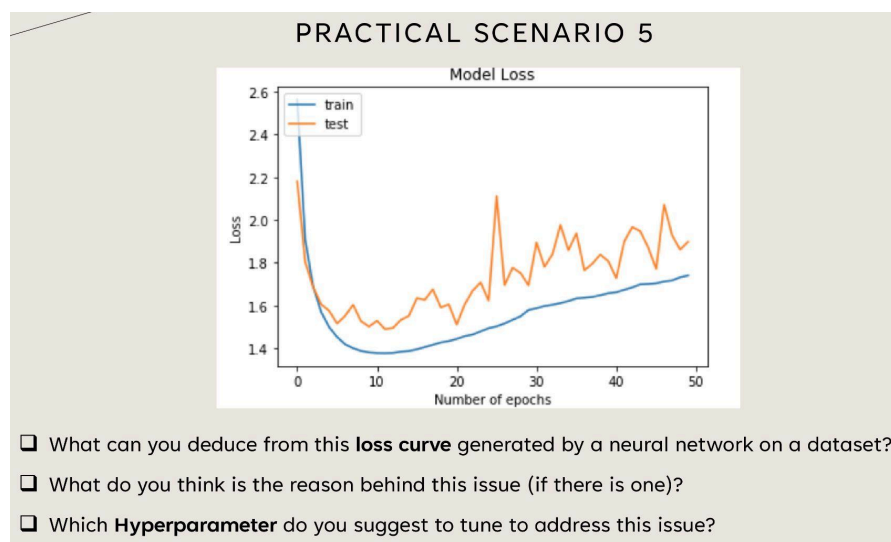
The polynomial regression model is too simple to capture the true patterns in the data. As a result, even with more data, the model cannot learn better relationships and performance remains limited.

3. Solutions to Fix Underfitting

Increase Model Complexity: Raise the degree of the polynomial to allow the model to capture non-linear patterns.

Feature Engineering: Create additional meaningful features from the existing data to improve learning capability.

Reduce Regularization (if applied): Lower the regularization strength (λ) so the model is less constrained and can fit the data better.



1. Observation from Loss Curve

The model is overfitting.

- Training and test loss decrease together at first
- After ~10–15 epochs, test loss starts increasing
- Training loss remains low while test loss worsens

This shows the model fits training data well but generalizes poorly.

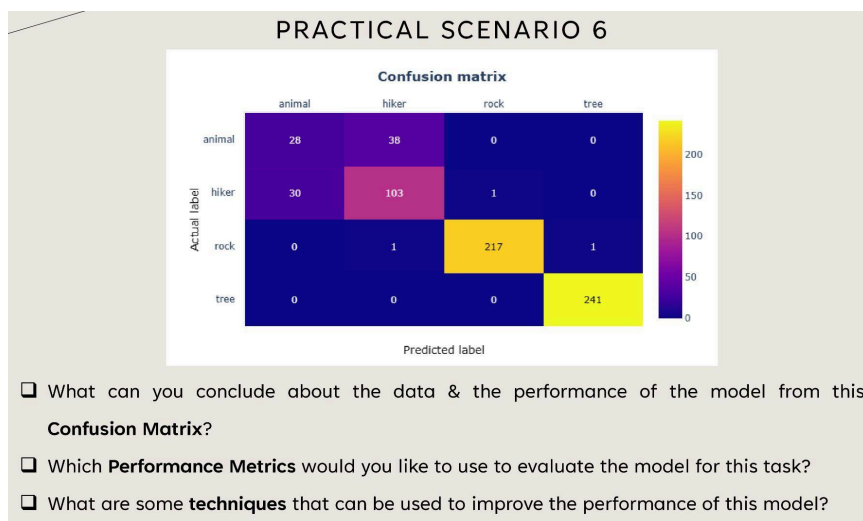
2. Reason for the Issue

The model is trained for too many epochs.

Instead of learning general patterns, the neural network starts memorizing training data (including noise and small fluctuations), which leads to poor performance on unseen data.

3. Hyperparameters to Fix the Issue

- Number of Epochs (Early Stopping):
Stop training around the point where validation loss stops improving ($\approx$ epoch 10).
- Regularization:
Reduce overfitting using:
 - L1 / L2 regularization (λ)
 - Dropout layers or dropout rate tuning
- Learning Rate (α):
Adjust learning rate to stabilize training and reduce volatility in loss curves.



1. Conclusions from the Confusion Matrix

- Class imbalance exists:
The dataset is uneven, with more samples of:
 - Trees (241)
 - Rocks (219)Compared to:
 - Hikers (134)
 - Animals (66)
- Model performance is uneven:
 - Very strong on trees and rocks (almost perfect predictions)

- Weak on animals vs hikers, showing confusion between the two
 - 38 animals predicted as hikers
 - 30 hikers predicted as animals

This shows the model is biased toward majority classes.

2. Appropriate Evaluation Metrics

Because the dataset is imbalanced, accuracy is not reliable.

Better metrics:

- Precision
- Recall
- F1-score
- AUC-ROC

These give a fair evaluation across all classes, especially minority ones.

3. Ways to Improve Performance

- Oversampling: Increase animal and hiker samples (duplicate or synthetic generation)
- Undersampling: Reduce tree and rock samples (risk: losing useful data)
- Model Improvement: Use better feature engineering or a more powerful architecture for fine-grained classification, and tune model hyperparameters accordingly

PRACTICAL SCENARIO 7

Anger	Disgust	Fear	Happiness	Sadness	Surprise	Neutral
						
						
						

Here you can see some sample images from the **FER2013** Dataset for Facial Expression Recognition. It is one of the benchmark datasets for that task.

☐ What kind of **Data Augmentation** techniques would you like to use on this dataset?

☐ What kind of **Model** do you think would be suitable for this task?

1. Data Augmentation Techniques

To improve generalization, you can apply data augmentation by tuning:

- Rotation Angle: slightly rotate images to create new variations
- Noise Level: add small random noise to make the model more robust

Additional useful (image-specific) augmentations:

- Horizontal flipping (valid for faces)
- Small zoom in/out
- Brightness and contrast adjustment

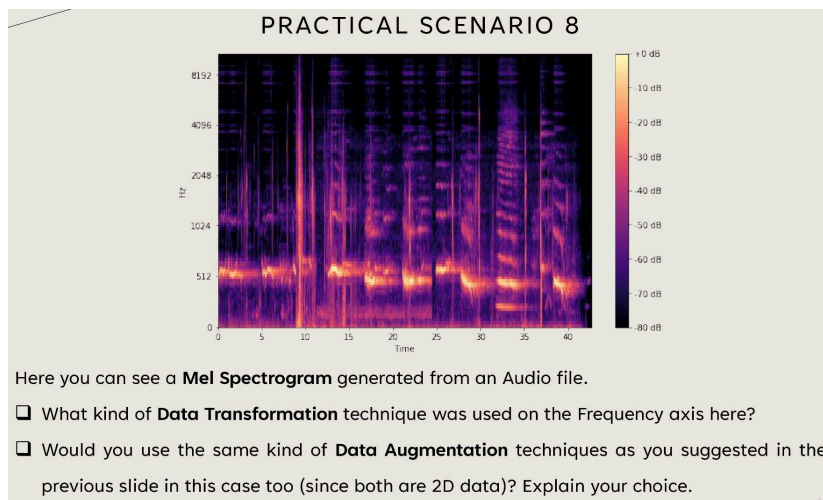
Avoid vertical flipping, since upside-down faces are not realistic and can confuse the model.

2. Suitable Model for the Task

The task involves facial image classification, so a Convolutional Neural Network (CNN) is the most suitable model.

- CNNs are designed for image data (2D pixel grids)
- They automatically learn spatial features like:
 - Eye shape
 - Mouth curvature
 - Facial edges and textures
- Model performance depends on tuning:
 - Number of layers
 - Architecture depth and complexity

CNNs are the standard choice for facial expression recognition tasks.



1. Data Transformation on the Frequency Axis

The spectrogram uses a logarithmic transformation on the frequency axis.

- The y-axis (Hz) increases in a log scale (e.g., 512, 1024, 2048, 4096)
- This compresses high-frequency values and expands low-frequency details

- The audio signal is first converted using a frequency-domain transformation (e.g., STFT) before visualization

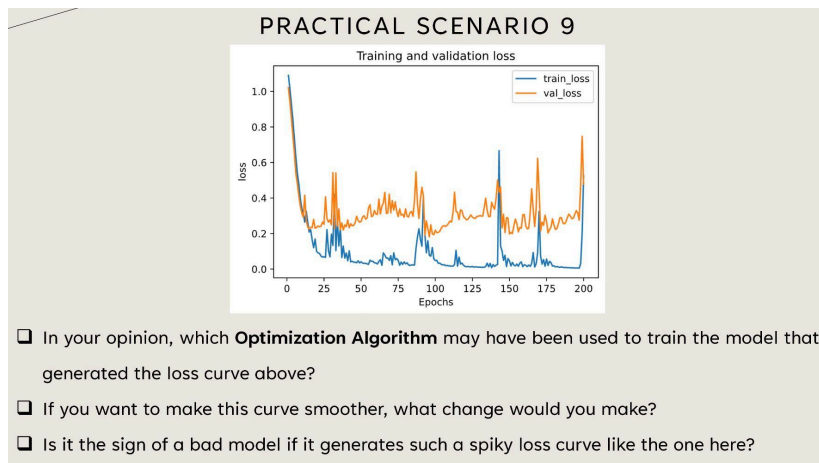
2. Suitable Data Augmentation Techniques

Standard image augmentations are not appropriate for spectrograms because axes have physical meaning:

- X-axis = Time
- Y-axis = Frequency

Incorrect transformations:

- Rotation → mixes time and frequency (invalid)
- Horizontal flip → reverses time (audio plays backward)
- Vertical flip → inverts pitch structure (unnatural audio)



1. Optimization Algorithm Used

The spiky and unstable loss curve suggests the use of Stochastic Gradient Descent (SGD) or mini-batch gradient descent with a small batch size.

- These methods update weights using small subsets of data
- This introduces high variance in updates
- As a result, the loss curve shows sharp fluctuations and noise

2. How to Make the Curve Smoother

To stabilize training, you can tune:

- Increase Batch Size:
Larger batches average gradients over more samples, reducing noise and making updates smoother.
- Decrease Learning Rate (α):
Smaller steps prevent large, unstable jumps during optimization.

3. Is This a Bad Model?

No, not necessarily.

- The overall trend shows loss decreasing, meaning the model is learning
- The spikes reflect noisy gradient updates, not failure

In fact, a certain level of noise can be useful because it helps the model avoid poor local minima and explore better solutions during training.